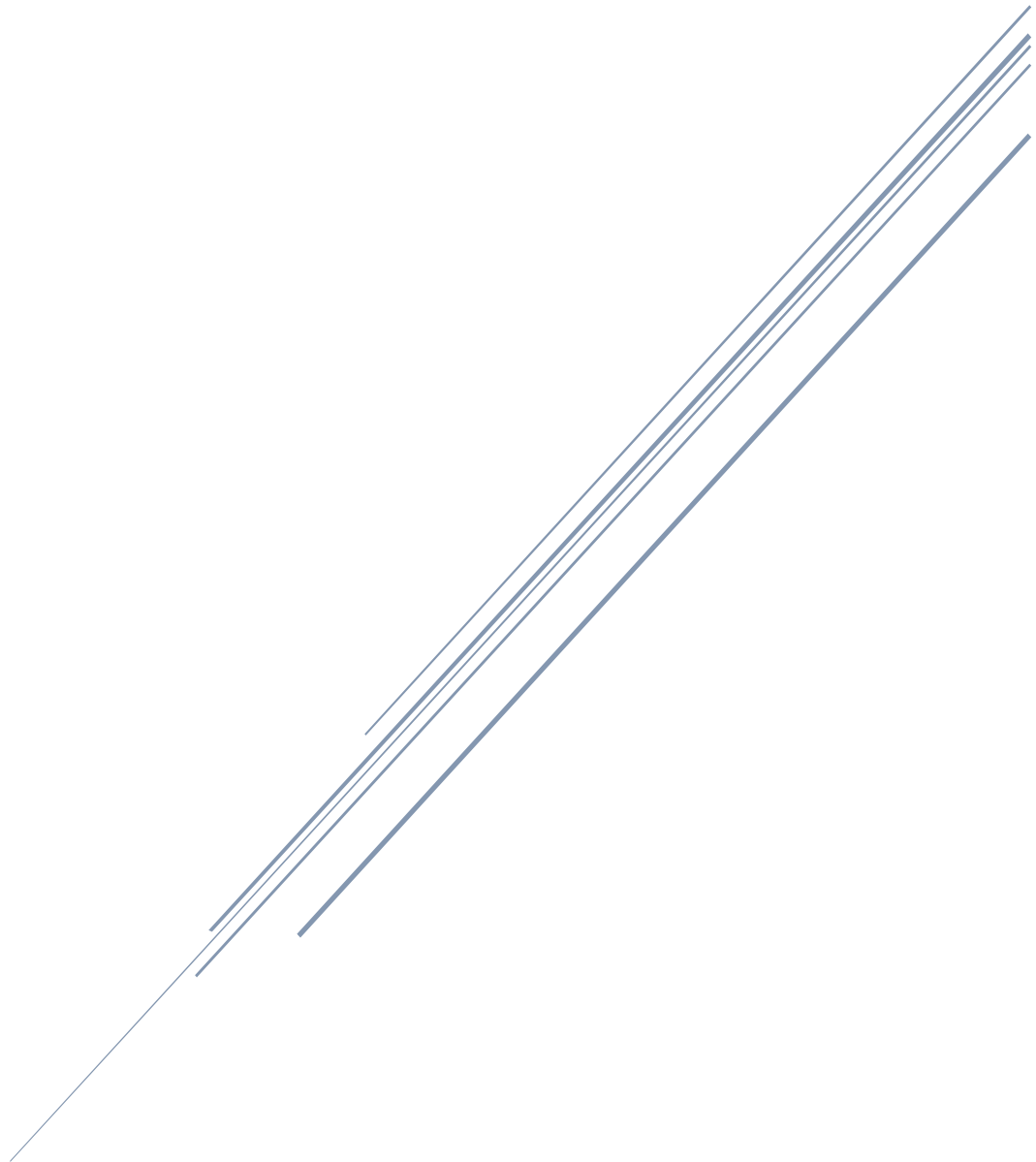


HOME SERVER

Documentation



Sobczak Mateusz

1.Contents

2.	Hardware.....	2
3.	Software.....	3
4.	Ubuntu Server 22.04.4 	4
a)	Apache2 	4
b)	Interpreter PHP 	4
c)	MySQL Server 	4
d)	phpMyAdmin 	4
e)	vsftpd 	5
1.	MSSQL Server 	5
2.	PostgreSQL 	6
3.	Remote access	7

2. Hardware

Server: HP T630

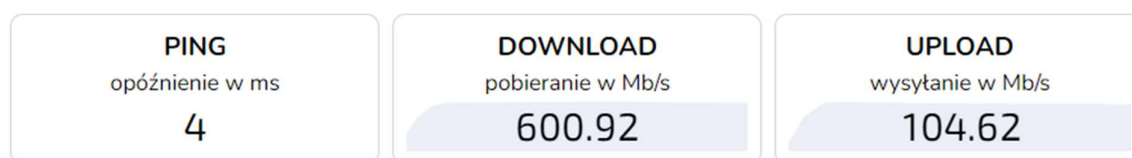
RAM: 8 GB

Disk: 256 GB SSD

Operating system: Proxmox Virtual Environment 7.4-17

Internet: Fiber optic T-mobile

Speed z <https://www.speedtest.pl/>

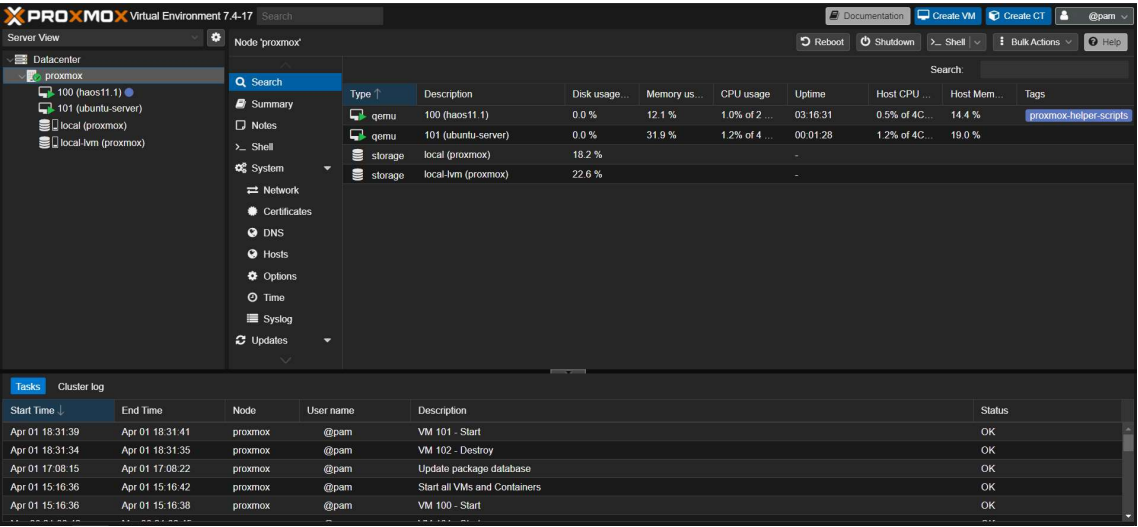


Router: Home Box Wi-Fi 6 Dual-Band (2.4 GHz/5 Ghz) Multigigabit Ethernet

Adressing IPv4, range 192.168.1.2 – 192.168.1.254

3. Software

The Proxmox Virtual Environment operating system version 7.4-17 is installed on the server. This system allows you to create and manage virtual machines with different operating systems, operating independently of each other. Currently, however, only one virtual machine is created called ubuntu-server with the identifier 101. This system is used for extensive future development possibilities, as well as for the ease of creating, managing and restoring backups. This system runs at the local address 192.168.1.66.



4. Ubuntu Server 22.04.4

It includes services for managing and sharing websites and web applications. It is located at the local address 192.168.1.200. The system was also installed to become a hosting server for websites and Internet applications in the future. Currently, it includes services such as:

a) Apache2

A service providing websites and web applications in version 2.4.58, which also allows you to configure the HTTPS encrypted module and add your own certificate. I used the solution of locating the website files in the "public_html" directory in the home directories of users with the name of a specific website/web application. These users have the "usr/sbin/nologin" shell set, so they cannot log in to the system terminal to manage it. This solution has the advantages mentioned in point e.

b) Interpreter PHP

A service, or more precisely, an extension to the Apache2 service, allowing the use of PHP version 8.2.4. Without this module, files with the .php extension would not work. This allows, among other things, to retrieve data from the database (using SQL), which is mainly why it was installed.

c) MySQL Server

The MySQL server service, which allows you to create and manage databases, is installed in version 8.3.0. Databases are managed in the system terminal using SQL queries. It also allows you to download data and manage it and databases from outside, for example using PHP (section b).

d) phpMyAdmin

A service that adds a web application with a graphical menu for database management (section c). During installation, you can select from the list a service that provides websites and Internet applications, in this case Apache2 (section a). This service was installed to facilitate database administration.

e) vsftpd

Service supporting FTP and sFTP version 3.0.5. It allows you to manage files located on the server's physical disk. This service also allows for many different configuration options. The configuration used supports the encrypted sFTP protocol with a self-signed certificate. This service has been installed and configured to allow specific users to upload files for their website or web application. For this purpose, in order to connect, you must authenticate as a specific user included in the list of users with this option (for example, the users from point a). These users only have access to their home directory, including the "public_html" directory. A maximum of 100 users can log in at a time, through the set range of ports.

5. MSSQL Server

On the next Ubuntu system is SQL Server. You can connect to it using SQL Server Management Studio. It also receives requests from applications, acts as an external server for local testing. It helps in creating applications on multiple devices, as you do not have to import the database on each one.

Tutorial:

First approach to the key director and registered in the location using commands:

```
curl -fsSL https://packages.microsoft.com/keys/microsoft.asc
```

Then you need to download and add the repositories with the command:

```
curl -fsSL https://packages.microsoft.com/config/ubuntu/22.04/mssql-server-2022.list
```

After completing these steps, it is required to refresh our repository list and download the MSSQL database engine. We can do this with these two commands:

```
sudo apt-get update
```

```
sudo apt-get install -y mssql-server
```

We can execute these commands as apt-get to be sure, or use apt instead of apt-get. The last step in installing the engine is to select the version we want to use and set the sa password using the command:

```
sudo /opt/mssql/bin/mssql-conf setup
```

We follow the instructions displayed in the console and enter the desired options. After completing these steps, we can check for sure whether the engine is already running with the command:

```
systemctl status mssql-server
```

The database engine will already be running, to ensure remote access we will open a port in the system firewall to be sure. We will do this with the command:

```
sudo ufw allow 1433/tcp
```

6. PostgreSQL

Also on Ubuntu. It is accessible from the local network for every user, while from outside it only for selected users. It handles requests from applications authenticated with the appropriate user. It also allows management from the pgAdmin application.

Tutorial:

First, you need to install the PostgreSQL database engine using the command:

```
sudo apt install postgresql
```

Then after installation we go to edit the engine configuration file with the command:

```
sudo nano /etc/postgresql/xx/main/postgresql.conf
```

where xx is the version number of your engine (e.g. 14). In the configuration file, find the line with `listen_addresses`, remove the first # to remove the comment, and after the = sign, enter the value you are interested in. '*' for the ability to connect from any host, 'localhost' for the ability to connect from the current device or a specific IP address/addresses.

```
#-----  
# CONNECTIONS AND AUTHENTICATION  
#-----  
  
# - Connection Settings -  
  
listen_addresses = '*'           # what IP address(es) to listen on;  
                                # comma-separated list of addresses;  
                                # defaults to 'localhost'; use '*' for all  
                                # (change requires restart)  
port = 5432                     # (change requires restart)  
max_connections = 100           # (change requires restart)
```

Here you can adjust the port in the port line, and the maximum number of active connections in the max_connections line

Save the file and exit

The next step is to edit the file that allows logging in the address, we will run the edit with the command

```
sudo nano /etc/postgresql/xx/main/pg_hba.conf
```

We move to the very bottom of the file and add the addresses there depending on whether we want to allow connections from an IPv4 or IPv6 address. To allow logins from an IPv4 address, we add "# IPv4 local connections:" under the entry in the header according to the pattern

```
# IPv4 local connections:  
host    all             all             127.0.0.1/32          scram-sha-256  
host    all             all             192.168.1.15/24       scram-sha-256
```

Where the first all means the database, so we can enter a specific one or all for each.

The second all means the user, so we can enter the username or all for each

The next value corresponds to the IP address. We can narrow it down to a specific address as in the example or to the entire network by entering 0 after the network prefix, it is important to add a network mask after /

The last value indicates the encoding method, we enter the same as in the already entered line.

For IPv6 we work analogously but under the heading "# IPv6 local connections:".

The last thing is to set the password for the postgres user. To do this, use the command:

```
sudo su postgres
```

```
psql -U postgres
```

After logging in to the database engine server, enter the command:

```
ALTER USER postgres WITH PASSWORD 'new_password';
```

Where new_password is the password we want to use to authenticate the postgres user when logging in.

Then we exit using the \q command. We return to the system user with the su command and restart the postgresql server with the command:

```
sudo systemctl restart postgresql
```

To connect from the outside, you must allow connections from the outside on the firewall, we must open port 5432 with the command:

```
sudo ufw allow 5432/tcp
```

7. Remote access

The server does not have an external IP, access from outside the network is carried out by the TailScale software. It creates a private network with tunneling, thanks to which each device has access to the others not from every location. TailScale also offers a funnel option, which allows you to launch a global domain address for a given local device. Thanks to this function, you can share websites and web applications, because it offers the possibility of sharing only port 443, 8443, 1000. I shared this function with phpMyAdmin, for friends to test.